

KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A++" Grade



KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

Customer Support Bot For Small Businesses

Project ID: PRJ083

Team ID: T083

Team GitHub Repository:

https://github.com/MdNaUsHadPY/HelpPilot-AI-.git

Student Name	Registration Number	Individual GitHub Link
MD NAUSHAD	99230041213	https://github.com/MdNaUsHadPY
ADITYA GUPTA	99230041222	https://github.com/adityagupta5git
MOHAMMAD SHAKIB	99230041214	https://github.com/mohammadshakib834-hub
ALBAKHSH SABIR	99230041209	https://github.com/albakhshsabir

Project Trainer / Guide

SHIRWIN PRINCE

Table of Contents

1.Introduction	3
2.Problem Statement & Objectives	3
3.System Architecture (RAG Pipeline)	3
4.Implementation Details & Tech Stack	4
5.Results and Testing	5
6.Conclusion & Future Scope	5

1. Introduction

Small businesses often receive similar questions from customers about their products, services, delivery, returns, business hours and support policies. Answering these questions one by one can take a lot of time, especially when the business does not have a separate customer support team.

To make this process easier, we developed **Customer Support Bot For Small Businesses**, named **HelpPilot AI**. It is a web-based chatbot that helps customers find answers using information provided by the business.

The application uses a technique called Retrieval-Augmented Generation (RAG). It searches through uploaded business documents, finds information related to the customer's question and uses that information to generate a suitable response through a chat interface.

2. Problem Statement & Objectives

Problem Statement:
Small businesses may not have enough staff or resources to handle every customer enquiry manually. Employees often need to go through different documents to find information about business policies, products or services. This can make the support process slow and sometimes lead to inconsistent answers.
A normal chatbot may also struggle to answer questions that depend on private or business-specific information. It may provide incorrect details if the required information is not available.

Objectives:

- To develop an AI-powered chatbot that can answer common customer-support questions.
- To provide an admin interface for uploading PDF, DOCX and TXT documents.
- To find relevant information from uploaded documents using semantic search.
- To generate clear and natural answers based on the retrieved information.
- To reduce incorrect or unsupported answers by guiding the AI to use the available business information.

3. System Architecture (RAG Pipeline)

The application follows two main workflows: document ingestion, performed when an administrator uploads business information, and question answering, performed when a customer sends a message. The frontend communicates with a Python FastAPI backend, which coordinates document processing, retrieval from Qdrant Cloud and response generation through Google's Gemini API.

Workflow

1. Document Ingestion Workflow (Admin)

Admin Uploads Document > Text Extraction > Text Chunking > Embedding Generation > Store Vectors in Qdrant Cloud

2. Question Answering Workflow (Customer)

Customer Asks Question > FastAPI Backend > Question Embedding > Search Relevant Chunks in Qdrant > Gemini AI Generates Answer > Display Answer to Customer

3.1 Data Ingestion Phase

The administrator uploads a PDF, DOCX or TXT file through the admin interface. The backend identifies the file type and extracts its readable text using PyPDF for PDF files, python-docx for DOCX files and text decoding for TXT files. The extracted text is divided into smaller chunks. Chunking makes large documents easier to process and retrieve, while overlap helps preserve context between neighbouring chunks.

Each document chunk is converted into a numerical vector using the Gemini Embedding API (gemini-embedding-001). The vectors, together with their text and metadata such as the original filename and chunk index, are stored in the Qdrant Cloud collection named support_docs.

3.2 Query & Generation Phase

When a customer submits a question, the frontend sends it to the FastAPI endpoint POST /api/chat. The question is converted into an embedding using the same embedding model, with the retrieval-query task type. Qdrant performs a similarity search and returns up to four relevant chunks (Top-K = 4), subject to the configured similarity threshold.

The backend combines the retrieved context, system instructions, the customer's question and recent conversation history. This prompt is sent to the configured Google Gemini generation model. The instructions tell the model to behave as a polite support assistant, use only the supplied context, avoid inventing business details and state when the requested information is unavailable. The generated answer and source filenames are returned to the frontend.

4. Implementation Details & Tech Stack

Component	Technology Used	Purpose
Programming Language	Python	Core backend logic and document processing
Backend Framework	FastAPI	Creates API endpoints and connects frontend with backend services
Frontend	HTML, CSS, JavaScript	Provides the customer chat and administrator interface
Vector Database	Qdrant Cloud	Stores and retrieves document embeddings using similarity search
Embedding Model	Gemini Embedding 001	Converts document chunks and questions into vectors
Large Language Model	Google Gemini API	Generates context-based customer-support responses
Document Processing	PyPDF, python-docx	Extracts text from PDF and DOCX documents
Deployment / Version Control	Vercel; Git and GitHub	Application deployment and source-code version control

4.1 Retrieval Configuration

The implementation retrieves up to four matching chunks for a customer query. A similarity threshold is applied to filter out weak matches. The chunk size, overlap and similarity threshold are controlled through the project's configuration; exact numeric values for these settings are not specified in the supplied project notes.

5. Results and Testing

The application workflow supports document upload, text extraction, chunk creation, embedding generation, vector storage, semantic retrieval and AI-based answer generation. The customer chat flow uses the retrieved business-document content as context for Gemini. When relevant information is not available in the retrieved content, the configured instructions require the assistant to say that the information is unavailable rather than inventing a policy, price, date or contact detail.

Functional testing areas:

- File handling: verify that PDF, DOCX and TXT files can be uploaded and their text extracted.
- Document ingestion: verify that extracted text is chunked, embedded and stored in the Qdrant collection.
- Semantic retrieval: ask questions using wording different from the source document and check whether relevant chunks are retrieved.
- Answer generation: verify that the response uses the retrieved context and remains concise and polite.
- Missing information: ask a question not covered by the uploaded documents and verify that the assistant does not invent an answer.
- Conversation context: test follow-up questions to check whether recent conversation history is considered.

Note: The supplied project notes do not include measured accuracy, response-time results or a completed test dataset. Such metrics should be added only after actual testing.

6. Conclusion & Future Scope

Conclusion: Customer Support Bot For Small Businesses (HelpPilot AI) provides a document-grounded approach to answering customer enquiries. By combining a web chat interface, FastAPI backend, Gemini embeddings, Qdrant semantic retrieval and Gemini response generation, the application can use business-specific documents to support customer interactions and reduce the need for staff to search manually for routine information.

Future Scope: Possible future improvements include adding user authentication and role-based access for business administrators, expanding document management features, supporting more document formats, adding multilingual responses, introducing customer feedback to evaluate answer quality, and monitoring retrieval accuracy and response time through structured testing.